

AGENTIC OS

Estudiante: Mario Alberto Alvarado Giler | Paralelo: 1

AGENTIC OS V2 - Diagrama de Despliegue y Procesamiento
Launcher local, ventanas remotas, cola de oraciones, Loader y P hilos detectores en IA Learner

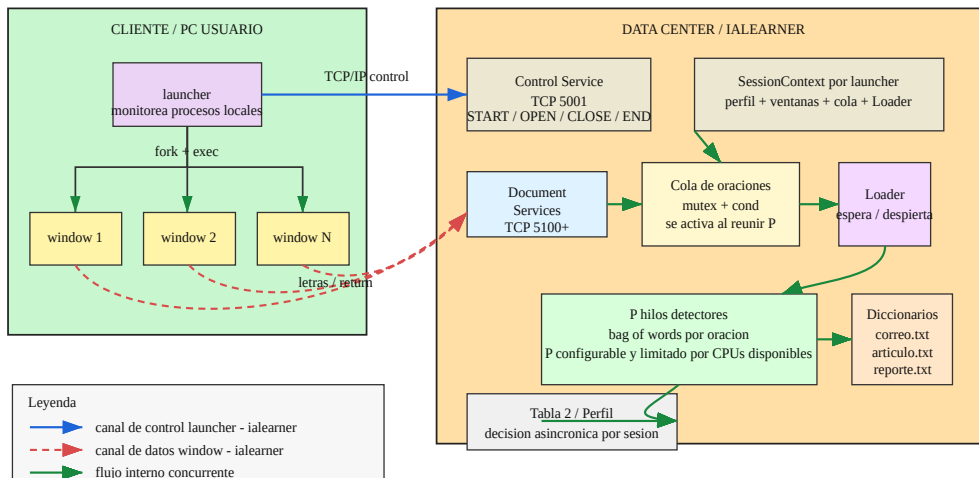


Figura 1. Diagrama actualizado de despliegue y procesamiento concurrente

Descripcion General

El sistema se resolvió con una arquitectura cliente-servidor. **ialearner** representa el data center, mientras que **launcher** y **window** representan el lado del usuario. El launcher administra solamente procesos locales: crea ventanas, cierra ventanas, muestra su estado y finaliza la sesión. Las ventanas son procesos gráficos independientes creados mediante *fork* y *exec*; capturan teclas con X11 y envían caracteres al servidor central por sockets TCP.

ialearner escucha conexiones de control en el puerto 5001. Cuando un launcher solicita una ventana, el data center asigna un puerto desde 5100 y crea un servicio de documentos para esa ventana. Esta asignación centralizada evita conflictos entre launchers o ventanas. Cada launcher crea una sesión independiente con su propio perfil, cola de oraciones y ventanas asociadas.

Procesamiento Concurrente

La segunda versión ya no espera al cierre de una ventana para clasificar texto. ialearner recibe caracteres continuamente y cada hilo de ventana acumula letras hasta formar una oración. La oración termina con *return* y se inserta en una cola protegida por mutex dentro de la sesión del launcher.

El parámetro **P**, por ejemplo `./ialearner 2`, define cuántas oraciones deben reunirse antes de activar la detección. Un hilo Loader duerme con una variable de condición y despierta cuando la cola alcanza **P** oraciones. Luego extrae el lote y lanza **P** hilos detectores, uno por oración. El coordinador espera con *pthread_join*. Si **P** supera los CPUs disponibles, se reduce para evitar sobrecarga.

La sincronización usa mutex y variables de condición: el perfil, la cola, la asignación de puertos y el procesamiento de lotes tienen protecciones separadas. Así se evita que el Loader y el cierre de sesión consuman la cola simultáneamente, y se reducen condiciones de carrera mientras varias ventanas generan oraciones.

Clasificación y Perfil

La clasificación usa *bag of words*: cada oración se convierte a minúsculas, se tokeniza y se compara con diccionarios de correo, artículo y reporte. Una clase es candidata si aparecen al menos tres palabras de su diccionario; si hay varias candidatas, gana la de mayor coincidencia. Esto evita clasificaciones por ruido.

Contexto detectado	Tipo de usuario
Correo electrónico	Personal administrativo
Correo electrónico + Reporte	Personal técnico
Correo electrónico + Artículo científico	Profesor
Artículo científico + Reporte	Estudiante
Correo electrónico + Artículo científico + Reporte	No detectado

El perfil se actualiza asincrónicamente después de cada lote. El sistema acumula las clases detectadas y aplica estrictamente la tabla de decisión. Si en una sesión aparecen correo, artículo y reporte, no se infiere usuario porque esa

combinacion no esta definida. Al finalizar, ialearnner procesa pendientes y envia el perfil al launcher por el canal de control.

Robustez

El programa valida sockets, memoria, hilos y cierre de recursos. Si una ventana se cierra, se procesa el texto pendiente antes de liberar el cliente. Al terminar una sesion se cierran servicios, se despierta al Loader y se liberan hilos, sockets y estructuras dinamicas. Si ialearnner falla, el launcher cierra sus ventanas locales para evitar procesos huérfanos.

Cumplimiento De Requisitos

La solucion integra los temas principales del proyecto: multiprogramacion con procesos independientes para las ventanas, IPC mediante sockets TCP, concurrencia con pthreads, sincronizacion con mutex y variables de condicion, y TDAs apropiadas para las operaciones del sistema. Los diccionarios se almacenan en una tabla hash para busqueda eficiente, las oraciones se administran en una cola concurrente y los hilos se coordinan para procesar lotes de tamano P. Por esta razon, el sistema no solo clasifica texto, sino que simula un entorno distribuido donde un data center aprende contexto de usuario a partir de eventos generados por procesos remotos.